

Assignment: Eigenvalue and Eigenvector

Academic Integrity Declaration

Check the statement that applies and sign your name.

Check	Statement
V	I have NOT used LLM (ChatGPT, etc), any online materials, or other students' reports/codes to complete this assignment.
	I have NOT completed this assignment entirely on my own.

If you received any assistance **that violates the class policy**, you must explain it in detail. A penalty will be applied depending on the **extent and nature of the assistance**.

Failure to disclose the assistance, you will get Grade F.

Examples:

- Screenshots of LLM prompt,
- Explanation of assistance : who/how/what extent etc

By signing below, I confirm that I have honestly and truthfully answered the academic integrity declaration.

Date: 2025.11.05

Sign: 유채은

What you need to submit

Submit as a zip file: [Assignment_Eigenvalue_YuChaeun_22200476.zip](#)

- **Report** (pdf)
- **Src Codes:** (1) [Assignment_Eigenvalue_YuChaeun_22200476.cpp](#)
 (2) [myNP_22200476.h](#), [myNP_22200476.cpp](#)
 (3) [myMatrix_22200476.h](#), [myMatrix_22200476.cpp](#)

- Download the template code

- [Assignment_EigenValue_ID.cpp](#) ([download template code](#))

Datafile:

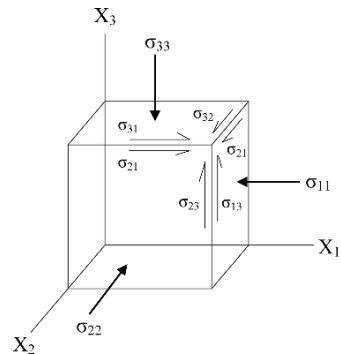
[Directory]

NP\NP_Data\Assignment7\

[File Name] (*.txt)

Question No.	Variable Name	File Name
Q1	Matrix matA_Q1	prob1_matA.txt
Q2	Matrix matA_Q2	prob2_matA.txt

Problem 1: The 3-D state of stress at a point is given by the stress tensor. Find the principal stresses and the principal directions at the point which are the **eigenvalues** and the **eigenvectors**, respectively.



$$\begin{bmatrix} \sigma_{ij} \end{bmatrix} = \begin{bmatrix} \sigma_{11} & \sigma_{12} & \sigma_{13} \\ \sigma_{21} & \sigma_{22} & \sigma_{23} \\ \sigma_{31} & \sigma_{32} & \sigma_{33} \end{bmatrix}$$

$$\sigma = A = \begin{bmatrix} 30 & 15 & 20 \\ 15 & 22 & 26 \\ 20 & 26 & 40 \end{bmatrix} \text{MPa}$$

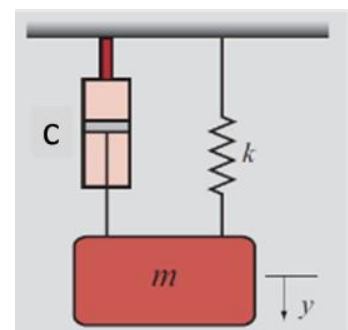
Problem 2: The characteristic equation of the 1-DOF system (mass-damper-spring system, m-c-k) is

$$m\ddot{y} + c\dot{y} + ky = 0$$

The displacement and velocity response of the system from an initial condition can be expressed as a form of

$$\mathbf{y} = c_1 e^{\lambda_1 t} \mathbf{v}_1 + c_2 e^{\lambda_2 t} \mathbf{v}_2$$

where $\mathbf{y} = \begin{bmatrix} y \\ \dot{y} \end{bmatrix}$, λ_1, λ_2 are the eigenvalues and $\mathbf{v}_1, \mathbf{v}_2$ are



the eigenvectors. The initial condition is at rest without stretching. Calculate the eigenvalues (λ_1, λ_2) and the eigenvectors ($\mathbf{v}_1, \mathbf{v}_2$) of the system to determine the response.

Assume, $m=10\text{kg}$; $k=800\text{ N/m}$; $c=200\text{ N/(m/s)}$.

HINT: The system eigenvalues can be found from the system matrix **A** where

$$\dot{\mathbf{y}} = \mathbf{A}\mathbf{y}$$

$$\begin{bmatrix} \dot{y} \\ \ddot{y} \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ -k/m & -c/m \end{bmatrix} \begin{bmatrix} y \\ \dot{y} \end{bmatrix}$$

Procedure

1. Real eigenvalues

a) Write a pseudocode for finding eigenvalue [5pt]

```
[lamda]= eig(A)
A1=A
for i=1 to N // or until Ai becomes U
    // Step1. QR decomposition (QiRi=Ai)
    [Q R] = QRHousehold (Ai);
    // Step2. Make into similar matrix form Ai+1 = QiTAiQi
    Ai+1 = RQ;
end U= Ai
// Step3 Eigenvalues from diagonal elements
lamda= diag(U) return
lamda
end of function

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%% [Q R] =

QRHousehold (Ai)
for j = 1:n-1

    // Step 1. Create vector [c]

    c = R(:, j);

    c(1:j-1) = 0;

    // Step 2. Create vector [e]

    e=zeros(n,1);

    e(j) = 1 ;

    if c(j) < 0    e(j) = -1 ;

    end

    // Step 3. Create vector [v]

    v = c + norm(c)*e;

    // Step 4. Create matrix [H]

    H = I-2.*v.*v' ./ (norm(v).^2) ;

    // Step 5. Update [Q], [R]

    Q = Q*H;

    R = H*R;

end
```

end

return Q, R end of
function

b) Create C functions of eig(A) that returns a vector of eigenvalues **[10pt]**

Matrix eigval(Matrix A);

- Input: matrix **A**(n x n)
- Output: vector nX1

- Must check if the input matrix is a square matrix.
 - Must check if there are any division by zero
-

// your code

```
Matrix eigval(Matrix A)
{
    // initialize
    int n = A.rows;
    int m = A.cols;
    if (m != n) {
        printf("Error!! A must be a square matrix!!\n");
        return A;
    }
    int itrMax = 100;
    Matrix U = copyMat(A);
    Matrix Q = eye(n, n);
    Matrix R = copyMat(A);
    Matrix lambdas = eye(n, 1);

    for (int a = 1; a <= itrMax; a++) {
        // Step 1 : A = QR decomposition
        QRdecomp(U, &Q, &R);

        // Step 2: Create Similar Matrix A = Q'AQ
        U = multMat(R, Q);
    }

    // Step 3: eigenvalues from U
    for (int b = 0; b < n; b++) {
        lambdas.at[b][0] = U.at[b][b];
    }
}
```

```

        // deallocation
        freeMat(U);          freeMat(Q);          freeMat(R);

    return lambdas;
}

////////////////////////////////////

void QRdecomp(Matrix U, Matrix* Q, Matrix* R)
{
    // Initialization
    int n = U.rows;
    Matrix I = eye(n, n);
    Matrix H = zeros(n, n);
    *R = copyMat(U);
    *Q = I;
    Matrix c = zeros(n, 1);
    Matrix e = zeros(n, 1);
    double c_norm, v_norm = 0;

    for (int k = 0; k < n; k++) {

        // Step 1. Create vector[c]
        for (int i = 0; i < n; i++) {
            c.at[i][0] = R->at[i][k];
        }
        for (int i = 0; i < k; i++) {
            c.at[i][0] = 0;
        }

        // step 1-1. c norm ----> c_norm == sqrt( |c|^2 ) == sqrt( c'c )
        Matrix cT = transpose(c);
        c_norm = sqrt(vTv(cT, c));

        // Step 2. Create vector[e]
        e = zeros(n, 1);
        e.at[k][0] = 1;
        if (c.at[k][0] < 0) {
            e.at[k][0] = -1;
        }

        // Step 3. Create vector[v]
        // (v_norm)^2 = v'v
        Matrix v = zeros(n, 1);
        v = addMat(c, smultMat(e, c_norm));
    }
}

```

```

// Step 4. Create matrix[H]
Matrix vT = transpose(v);
v_norm = vTv(vT, v);
H = subMat(I, smultMat(multMat(v, vT), (2 / v_norm))); // H = I -
2*v*vT/vTv

// Step 5. Update[Q], [R]
*Q = multMat(*Q, H);
*R = multMat(H, *R);

// deallocation in the for loop
freeMat(cT);
freeMat(v);          freeMat(vT);
}

// Deallocate Memory
freeMat(I);          freeMat(H);          //freeMat(R);          freeMat(Q);
freeMat(c);          freeMat(e);

}

```

2. Find Eigenvectors (Simple method for up to 3x3 matrix)

- Use the simple method learnt in class to find eigenvectors
- Assume, this function is for 2x2 ~ 3x3 matrix only.

a) Write a pseudocode for finding eigenvector **[5pt]**

```

Matrix eigvec(Matrix A)
{

```

```

    for (int k = 0; k < n; k++) { // k -> v=1,2,3
        Matrix B = A

        // step 1. create matrix B = A - lambda*I
        for i = 0 to n-1
            B(i, i) -= lambdas(k)
        End i

        // step 2. create (n-1)x(n-1) matrix b
        row_ = 0, col = 0;
        for i = 0 to n - 1
            if (i == k) {
                continue;
            }

```

```

    }

    for j = 0 to n-1
        if (j == k) {
            continue;
        }
        B(row_, col) = B(i, j)
        col++
    end j
    row_++
    col = 0
end i
row_ = 0

// step 3. get a matrix 'b_inv' & a vector 'vecb'
invMat(&b, &b_inv);

for a = 0 to n-1
    if (a == k)
        continue
    vecb(row_) = -B(a, k)
    row_++
end a

// step 4. get the vectors v123 by multMat(b_inv, b_minus)
tmpv = b_inv * vecb

idx = 0;
for y = 0 to n-1
    if (y == k)
        v3(y) = 1
    else
        v3(y) = tmpv(idx)
        idx ++
    end y

// step 5. normalizing
vnorm = sqrt(vTv)
v3 = (1.0 / vnorm) * v3

// step 6. get the eigenvector by gathering v123
for c = 0 to n-1
    v_final(c, k) = v3(c)
end c

end k

return v_final;
}

```

b) Create C function of eigvec(A) [10pt]

`Matrix eigvec(Matrix A);`

- Input: Matrix **A** (n x n)
- Output: Matrix **V** (nxn): columns of eigenvectors

- Must check if the input matrix is a square matrix, 2x2 or 3x3.
- Must check if there are any division by zero
- Use your own function for matrix inverse

```
Matrix eigvec(Matrix A)
{
    int n = A.rows;

    Matrix lambdas = eigval(A);
    Matrix v_final = zeros(n, n);

    for (int k = 0; k < n; k++) { // k -> v=1,2,3
        printf("-----\n");
        printf("k = %d\n", k);
        Matrix B = copyMat(A);
        // step 1. create matrix B = A - lambda*I
        for (int i = 0; i < n; i++) {
            B.at[i][i] -= lambdas.at[k][0];
        }
        printMat(B, "B");
        Matrix b = zeros(n - 1, n - 1);
        Matrix b_inv = zeros(n - 1, n - 1);
        Matrix vecb = zeros(n - 1, 1);
        Matrix tmpv = zeros(n - 1, 1);
        Matrix v3 = zeros(n, 1);

        // step 2. create (n-1)x(n-1) matrix b
        int row_ = 0, col = 0;
        for (int i = 0; i < n; i++) {
            if (i == k) {
                continue;
            }

            for (int j = 0; j < n; j++) {
                if (j == k) {
                    continue;
                }
                b.at[row_][col] = B.at[i][j];
                col++;
            }
            row_++;
        }
    }
}
```

```

        }
        row_++;
        col = 0;
    }
    row_ = 0;
    printMat(b, "b1");

    // step 3. get a matrix 'b_inv' & a vector 'vecb'
    invMat(&b, &b_inv);
    printMat(b_inv, "binv");

    for (int a = 0; a < n; a++) {
        if (a == k)
            continue;
        vecb.at[row_][0] = -B.at[a][k];
        row_++;
    }
    printMat(vecb, "vecb");

    // step 4. get the vectors v123 by multMat(b_inv, b_minus)
    tmpv = multMat(b_inv, vecb);
    printMat(tmpv, "v2");

    int idx = 0;
    for (int y = 0; y < n; y++) {
        if (y == k)
            v3.at[y][0] = 1.0;
        else
            v3.at[y][0] = tmpv.at[idx++][0];
    }
    printMat(v3, "v3");

    // step 5. normalizing
    Matrix vT = transpose(v3);
    double vnorm = sqrt(vTv(vT, v3));

    v3 = smultMat(v3, 1.0 / vnorm);

    // step 6. get the eigenvector by gathering v123
    for (int c = 0; c < n; c++) {
        v_final.at[c][k] = v3.at[c][0];
    }

    // Deallocation in for loop
    freeMat(B);           freeMat(b);           freeMat(b_inv);
    freeMat(vecb);        freeMat(tmpv);        freeMat(v3);
    freeMat(vT);

} // end k

```

```
freeMat(lambdas);
```

```
return v_final;
```

```
}
```

3. Create C function that returns eigenvalue and eigenvectors [5 pt]

`Matrix eig(Matrix A, Matrix V, Matrix D);`

- Input: Matrix **A** (n x n)
- Output: Matrix **V** (nxn): columns of eigenvectors
- Output: Matrix **D** (nxn): diagonal matrix D of eigenvalues

(Important)

- Eigenvalues in matrix D should be in descending order (Highest to lowest)
- e.g. The highest value of eigenvalue should be the 1st element of matrix **D**
- Matrix V whose columns are the corresponding to eigenvalues.

```
double eig(Matrix A, Matrix* V, Matrix* D)
{
    int n = A.rows;
    int m = A.cols;
    if (n != m) {
        printf("Dimension ERROR!!!\n\n");
        return 1;
    }

    *V = eigvec(A);
    Matrix lambdas = eigval(A);

    // sorting in a descending order
    double tmp = 0;
    Matrix tmpv = createMat(n, 1);
    for (int i = 0; i < n - 1; i++) {
        for (int j = i + 1; j < n; j++) {
            if (lambdas.at[i][0] < lambdas.at[j][0]) {
                tmp = lambdas.at[i][0];
                for (int r = 0; r < n; r++) {
                    tmpv.at[r][0] = V->at[r][i];
                }

                lambdas.at[i][0] = lambdas.at[j][0];
                for (int r = 0; r < n; r++) {
                    V->at[r][i] = V->at[r][j];
                }
                lambdas.at[j][0] = tmp;
                for (int r = 0; r < n; r++) {
                    V->at[r][j] = tmpv.at[r][0];
                }
            }
        } // end j
    } // end i

    *D = zeros(n, n);
```

```

    for (int z = 0; z < n; z++) {
        D->at[z][z] = lambdas.at[z][0];
    }

    // Deallocation memory
    freeMat(lambdas);      freeMat(tmpv);
}

```

4. Show the output results for eigenvalues and vectors [5pt]

Q1.

Q1.

[Eigen value]

Eigenvalue Q1 =

```

73.031016
15.507761
3.461223

```

[Eigen vector]

Eigenvector Q1 =

```

0.502278      -0.863751      0.040641
0.505193      0.254981     -0.824479
0.701782      0.434649      0.564432

```

V_Q1 =

```

0.502278      -0.863751      0.040641
0.505193      0.254981     -0.824479
0.701782      0.434649      0.564432

```

D_Q1 =

```

73.031016      0.000000      0.000000
0.000000      15.507761      0.000000
0.000000      0.000000      3.461223

```

Q2.

```

- - - - -
Q2.

[Eigen value]

Eigenvalue Q2 =
    -14.472136
     -5.527864

[Eigen vector]

Eigenvector Q2 =
     0.068934    -0.178012
    -0.997621     0.984028

V_Q2 =
    -0.178012     0.068934
     0.984028    -0.997621

D_Q2 =
    -5.527864     0.000000
     0.000000    -14.472136

Press any key to continue . . . |

```

5. Compare your answer with MATLAB [5pt]

Write Matlab code

```

// your MATLAB code
// use [V, D]=eig(A)

clear all, close all, clc ;

A = [30    15    20;
     15    22    26;
     20    26    40];
B = [0     1;
     -80  -20];

[V1, D1]=eig(A);
[V2, D2]=eig(B);

fprintf('----- Q1 -----\n');
fprintf(['eigenvectors (V)\n']);
disp(V1);
fprintf('\n');
fprintf(['eigenvalues (D)\n']);
disp(D1);

fprintf('----- Q2 -----\n');

```

```

fprintf(['eigenvectors (V)\n']);
disp(V2);
fprintf('\n');
fprintf(['eigenvalues (D)\n']);
disp(D2);

```

// MATLAB output

```

----- Q1 -----
eigenvectors (V)
    -0.0406    0.8638    0.5023
     0.8245   -0.2550    0.5052
    -0.5644   -0.4346    0.7018
eigenvalues (D)
     3.4612         0         0
         0    15.5078         0
         0         0    73.0310

----- Q2 -----
eigenvectors (V)
     0.1780   -0.0689
    -0.9840    0.9976
eigenvalues (D)
    -5.5279         0
         0   -14.4721

```

Appendix

- A simple method of calculating the eigenvector can be as follows.

After finding the eigenvectors, we are interested in finding the eigenvector

v : $(A-\lambda I)v=0$

Let $B=(A-\lambda I)$, and A, B are 3 by 3 matrix Let the eigenvectors are initially set as

$$V_1 = \begin{bmatrix} 1 \\ v_{12} \\ v_{13} \end{bmatrix}, \quad V_2 = \begin{bmatrix} v_{22} \\ 1 \\ v_{23} \end{bmatrix}, \quad V_3 = \begin{bmatrix} v_{31} \\ v_{32} \\ 1 \end{bmatrix}$$

For the first eigenvector V_1

$$\begin{bmatrix} b_{11} & b_{12} & b_{13} \\ b_{21} & b_{22} & b_{23} \\ b_{31} & b_{32} & b_{33} \end{bmatrix} \begin{bmatrix} 1 \\ v_{12} \\ v_{13} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}, \quad \begin{bmatrix} b_{22} & b_{23} \\ b_{32} & b_{33} \end{bmatrix} \begin{bmatrix} v_{12} \\ v_{13} \end{bmatrix} = \begin{bmatrix} -b_{21} \\ -b_{31} \end{bmatrix}$$

If the inverse matrix exists, then

$$\begin{bmatrix} v_{12} \\ v_{13} \end{bmatrix} = \begin{bmatrix} b_{22} & b_{23} \\ b_{32} & b_{33} \end{bmatrix}^{-1} \begin{bmatrix} -b_{21} \\ -b_{31} \end{bmatrix}$$

Normalize the eigenvector V_1

$$V_1 = \frac{V_1}{\|V_1\|}$$

For the other eigenvector of V_2 ,

$$\begin{bmatrix} b_{11} & b_{12} & b_{13} \\ b_{21} & b_{22} & b_{23} \\ b_{31} & b_{32} & b_{33} \end{bmatrix} \begin{bmatrix} v_{22} \\ 1 \\ v_{23} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}, \quad \begin{bmatrix} b_{11} & b_{13} \\ b_{31} & b_{33} \end{bmatrix} \begin{bmatrix} v_{12} \\ v_{13} \end{bmatrix} = \begin{bmatrix} -b_{12} \\ -b_{32} \end{bmatrix}$$

Repeat for V_3